



Lab 8

Sequential Logic

Prof. Chih-Cheng Tseng (曾志成)

cctseng@mail.ntou.edu.tw

Contents of this slide are provided by NTOU VLSI Lab601

Objectives

- Demonstrate the designs of
 - D Flip-Flop (DFF)
 - 2-bit Counter
- Develop and simulate 3-bit and 4-bit counters by using SystemVerilog.
- Verify the designs by the DE0.

Designing a D Flip-Flop (DFF)

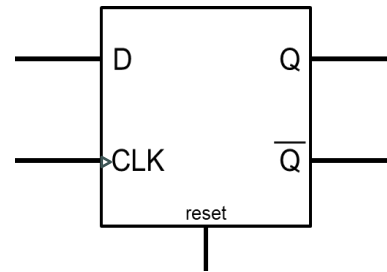
• Sample codes

```
module dff_pos_aync( // Positive-edge triggered DFF with asynchronous reset
    input d,
    input clk,
    input rst,
    output logic q,
    output logic q_n
);

    always_ff @ (posedge clk or posedge rst) begin
        if (rst) begin
            q <= 1'b0;
            q_n <= 1'b1;
        end
        else begin
            q <= d;
            q_n <= ~d;
        end
    end
endmodule
```

```
module dff_pos_sync( // Positive-edge triggered DFF with synchronous reset
    input d,
    input clk,
    input rst,
    output logic q,
    output logic q_n // outputting the inverted value
);

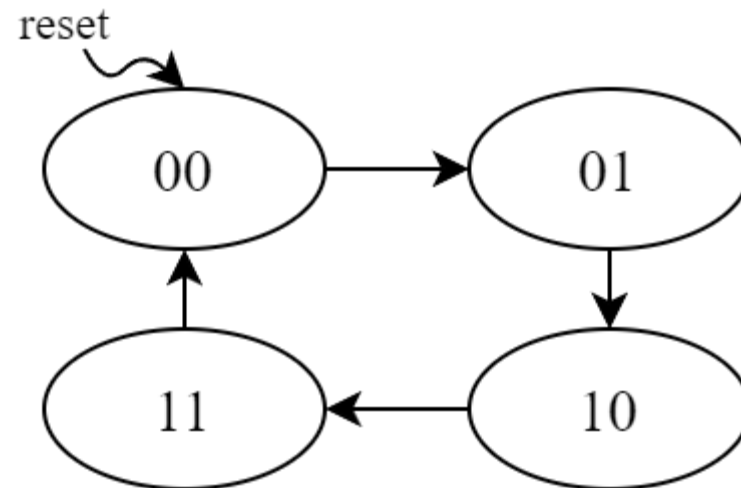
    // DFF behavior description
    always_ff @ (posedge clk) begin // The parentheses indicate the triggering condition
        if (rst) begin // When the positive edge of clk occurs and reset = 1
            q <= 1'b0; // q = 0, q_n = 1
            q_n <= 1'b1;
        end
        else begin // When the positive edge of clk occurs and reset = 0
            q <= d; // q = d, q_n = d inverted
            q_n <= ~d;
        end
    end
endmodule
```



- **always_ff @ (trig_cond)**
 - a SystemVerilog-specific construct used to describe sequential logic (flip-flops)
 - **always_ff** is a SystemVerilog **keyword** that ensures you only describe flip-flop behavior enclosed between the begin...end block.
 - **trig_cond** defines the trigger condition
 - posedge or negedge

Designing a 2-bit Synchronous Counter (1/5)

- Design a 2-bit synchronous counter
- Signal settings
 - Input: *clk* and *rst*
 - Output: counter [1:0]
- State diagram



Designing a 2-bit Synchronous Counter (2/5)

- State Table

Present State (t)		Next State ($t + 1$)	
$A(t)$	$B(t)$	$A(t + 1)$	$B(t + 1)$
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	0

- K-Map

A \ B	0	1
0	0	1
1	1	0

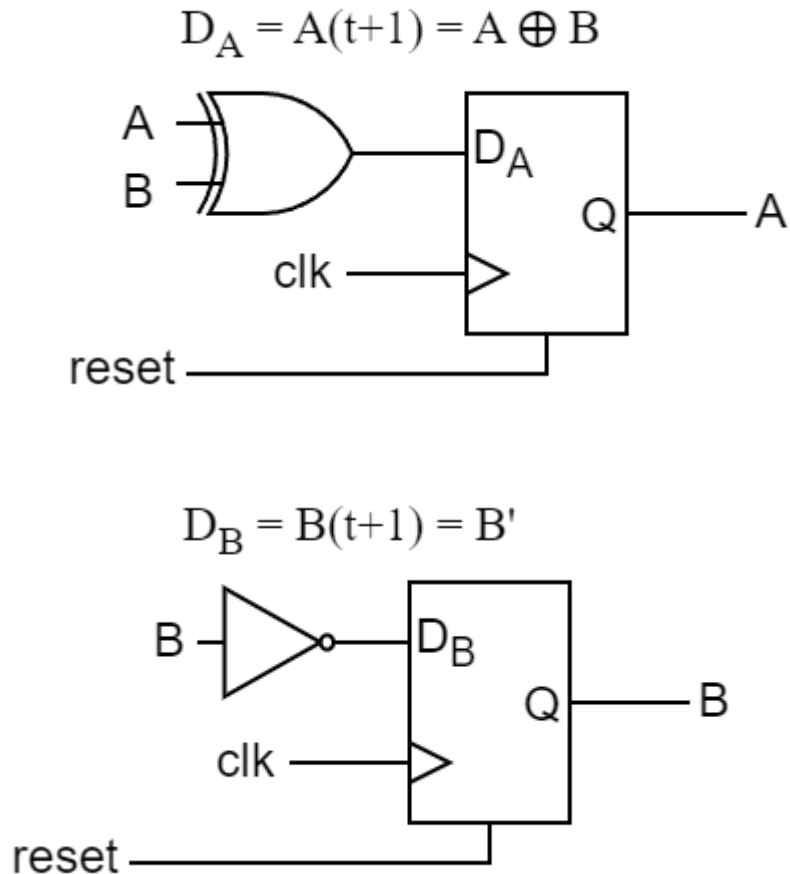
$$D_A = A(t+1) = A \oplus B$$

A \ B	0	1
0	1	0
1	1	0

$$D_B = B(t+1) = B'$$

Designing a 2-bit Synchronous Counter (3/5)

• Circuit



• Sample Code

```
module counter_2bits(
    input clk,
    input rst,
    output logic [1:0] count
);
    logic a, b, da, db;

    //----- call DFF -----
    dff_pos_sync u_dff_pos_sync_0(
        .d(da),
        .clk(clk),
        .rst(rst),
        .q(a)
    );

    dff_pos_sync u_dff_pos_sync_1(
        .d(db),
        .clk(clk),
        .rst(rst),
        .q(b)
    );

    //----- combinational Logic (draw K-Map) -----
    assign da = a ^ b;
    assign db = ~b;
    assign count = {a, b};

endmodule
```

Designing a 2-bit Synchronous Counter (4/5)

- Testbench

```
module test_counter_2bits;

    logic [1:0] count;
    logic clk, rst;

    counter_2bits u_counter_2bits(
        .clk(clk),
        .rst(rst),
        .count(count)
    );

    always #5 clk = ~clk;    // clk inverts its state every 5 ns

    initial begin
        clk = 0;  rst = 1;    // set clk = 0 and reset the DFF
        #10 rst = 0;          // FSM start
        #500 $stop;          // 500ns stop
    end

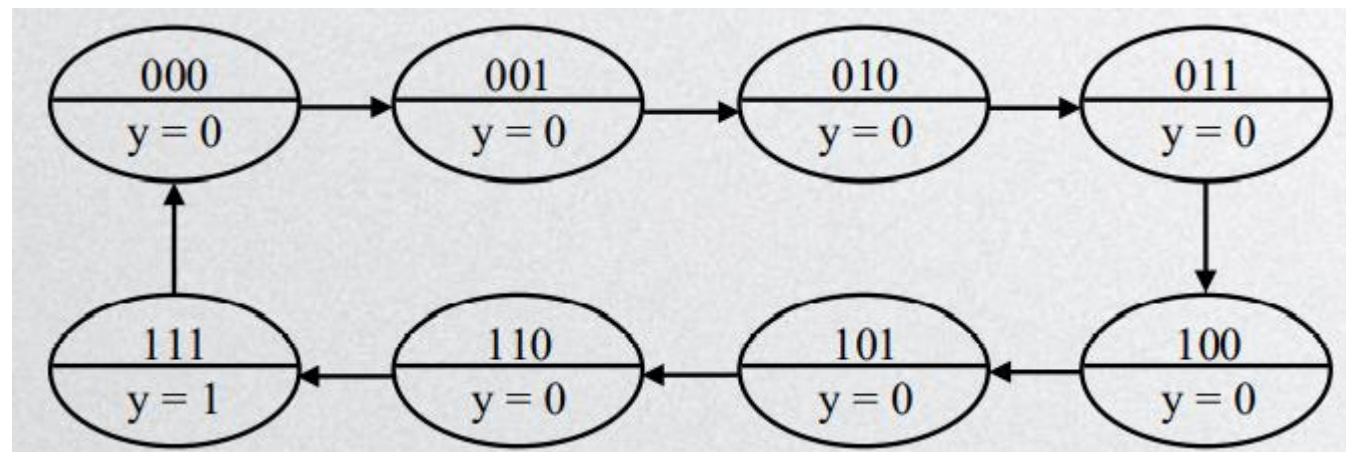
endmodule
```

- mcu.sv

```
counter_2bits u_counter_2bits(
    .clk(BTN[0]),
    .rst(~BTN[1]),    // button press = 0
    .count(LED[1:0])
);
```


LAB 1 - 3-bit Synchronous Counter (1/5)

- Design a 3-bit synchronous counter with an output y .
- Performed signal simulation and downloaded to DE0 for verification.
- Signal settings:
 - Input: clk and rst
 - Output: $Q[2:0] = \{A, B, C\}$ and y
- State diagram:



LAB 1 - 3-bit Synchronous Counter (2/5)

- State Table

Present State (t)			Next State ($t + 1$)			Output
$A(t)$	$B(t)$	$C(t)$	$A(t + 1)$	$B(t + 1)$	$C(t + 1)$	y
0	0	0	0	0	1	0
0	0	1	0	1	0	0
0	1	0	0	1	1	0
0	1	1	1	0	0	0
1	0	0	1	0	1	0
1	0	1	1	1	0	0
1	1	0	1	1	1	0
1	1	1	0	0	0	1

LAB 1 - 3-bit Synchronous Counter (3/5)

- Testbench

```

module test_counter_3bits;

    logic [2:0] q;
    logic y;
    logic clk, rst;

    counter_3bits counter_3bits(
        .clk(clk),
        .rst(rst),
        .q(q),
        .y(y)
    );

    always #5 clk = ~clk;    // clk inverts its state every 5 ns

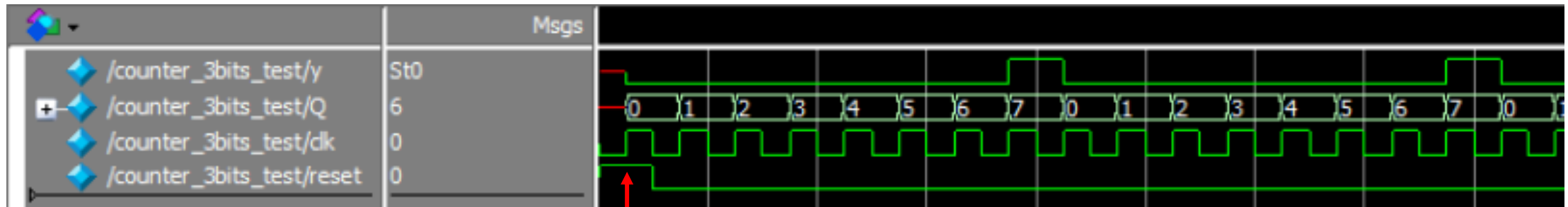
    initial begin
        clk = 0; rst = 1;    // set clk = 0 and reset the DFF
        #10 rst = 0;         // FSM start
        #500 $stop;         // 500ns stop
    end

endmodule

```

LAB 1 - 3-bit Synchronous Counter (4/5)

- Waveform



reset

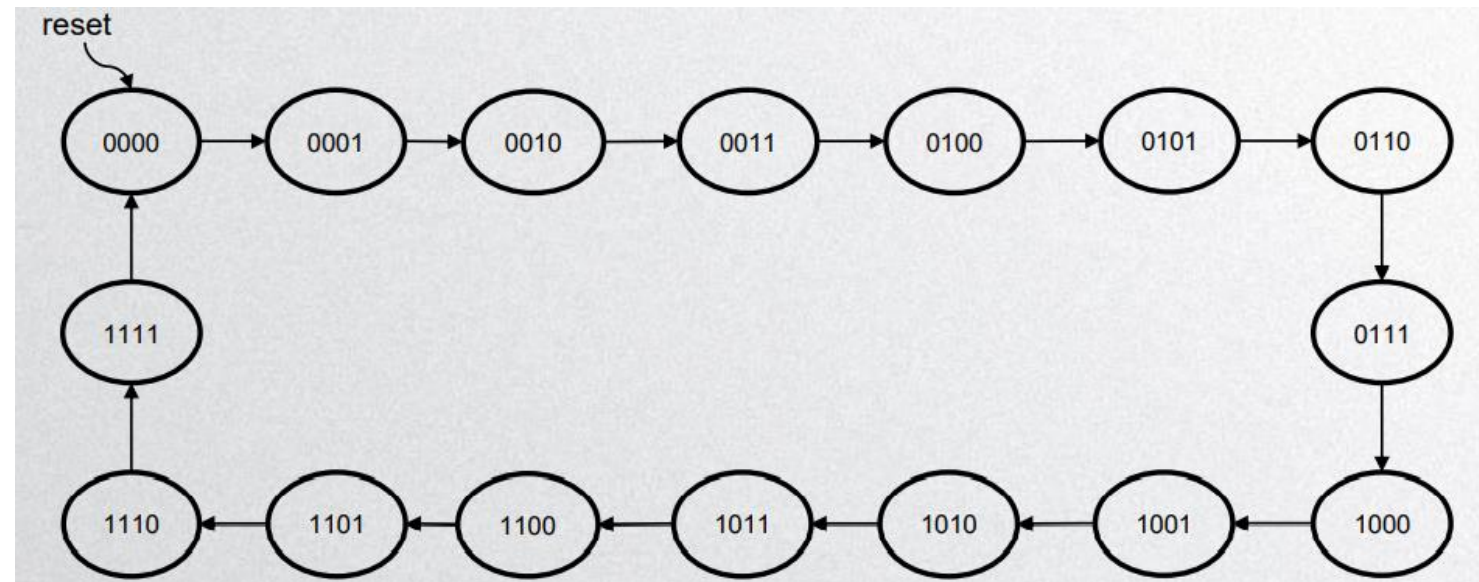
LAB 1 - 3-bit Synchronous Counter (5/5)

- Signals and pins setting

Signal	Type	DE0 pin
<i>clk</i>	Input	Button (BTN[0])
<i>rst</i>		Button (BTN[1])
<i>Q</i>	Output	LED (LED[2:0])
<i>y</i>		LED (LED[3])

LAB 2 - 4-bit Synchronous Counter (1/4)

- Design a 4-bit synchronous counter and connect its output to an SSD.
- Performed signal simulation and downloaded to DE0 for verification.
- Signal settings:
 - Input: *clk* and *rst*
 - Output: *seg[6:0]*
- State diagram:



LAB 2 - 4-bit Synchronous Counter (2/4)

- State Table

Present State (t)				Next State ($t + 1$)			
$A(t)$	$B(t)$	$C(t)$	$D(t)$	$A(t + 1)$	$B(t + 1)$	$C(t + 1)$	$D(t + 1)$
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	1	0	1	0
1	0	1	0	1	0	1	1
1	0	1	1	1	1	0	0
1	1	0	0	1	1	0	1
1	1	0	1	1	1	1	0
1	1	1	0	1	1	1	1
1	1	1	1	0	0	0	0

LAB 2 - 4-bit Synchronous Counter (3/4)

- Testbench

```

module test_counter_4bits;

    logic [3:0] q;
    logic clk, rst;

    counter_4bits counter_4bits(
        .clk(clk),
        .rst(rst),
        .q(q)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;  rst = 1;
        #10 rst = 0;
        #500 $stop;
    end

endmodule

```

LAB 2 - 4-bit Synchronous Counter (4/4)

- Signals and pins setting

Signal	Type	DE0 pin
<i>clk</i>	Input	Button (BTN[0])
<i>rst</i>		Button (BTN[1])
<i>seg</i>	Output	Seven Segment Display (HEX0)

- The output “seg” must be connected to a decoder before connecting to the SSD.